# Implicit Generation of the Big Six

# 编译器隐式合成六大件的总结

本文基于 **C++11 及之后标准**，精简梳理类中六大特殊成员函数的隐式自动合成与禁用条件。

## 一、六大特殊成员函数（The Big Six）

1. 默认构造函数： `T()`
2. 析构函数： `~T()`
3. 拷贝构造函数： `T(const T&)`
4. 拷贝赋值运算符： `T& operator=(const T&)`
5. 移动构造函数： `T(T&&)`
6. 移动赋值运算符： `T& operator=(T&&)`

## 二、关键要点

> 注：下文提到的“操作”=“构造函数+赋值运算符”
> 比如：拷贝操作，指的是：拷贝构造函数+拷贝赋值运算符

1. **六大件都满足的总原则！！！**
   - 只要是用户已经显式声明的六大件，就不会再被隐式自动合成。这是显然的。所以表格第三列的每一行第一种情形均为自己本身被声明。
   - 其实，我们真正需要关注的是：我们没有显式声明的函数，会不会被编译器隐式自动合成。
   - 需要额外花精力同时也是核心的是：**部分函数能否隐式合成还会受到他者是否显式声明的影响**。
2. **拷贝操作受移动操作影响**
   - 若显式声明移动构造或移动赋值，拷贝构造与拷贝赋值无法被隐式合成，无法使用。
   - **拷贝操作内部不互斥**（何为操作内部互斥，请看下一条）
3. **移动操作最为严格**

- 只要开发者显示声明了**拷贝、移动、析构**中任意一个，编译器均不会自动生成移动构造与移动赋值。
- **移动操作内部互斥**。不仅拷贝操作和析构函数会抑制移动操作，移动构造函数和移动赋值运算符也会相互抑制。一旦用户显式声明了其中一个，另一个移动操作就不会被自动合成。但是拷贝操作内部不会互斥，也就是说，显示声明拷贝构造函数，并不会抑制拷贝赋值运算符的隐式合成。

4. **隐式合成行为均为浅拷贝/浅处理**
   - 编译器合成的拷贝、移动操作均为逐成员浅处理，持有指针、堆内存（使用关键词 `new` 申请的内存）等资源的类必须手动实现。
5. **`= default` / `= delete` 的影响**
   - C++ 标准明确规定：
     用户声明（user-declared）包含三种情况：
     a. 手动实现（写函数体）
     b. = default
     c. = delete
   - 显式写 `= default`；视为用户手动显式声明，会影响其他函数的合成。
   - 例如，如果写了 `拷贝构造函数 = default` 会阻止移动函数的隐式合成
   - 显式写 `= delete` 同样视为用户显式声明，只不过是先声明再删除，也会影响其他函数的合成。
   - 例如，如果写了 `拷贝构造函数 = delete` 也同样会阻止移动函数的隐式合成
6. **`std::move` 本身不移动资源**
   - **`std::move` 的唯一作用是：将左值转换为右值引用 `T&&`**。不窃取资源，真正实现资源转移的是移动函数。也就是说，如果我对一个实例化对象使用了 `std::move`，但是却没有将返回值交给任何其他同类对象，那么此对象保持原样，没有任何资源被窃取。~~`std::move`：小丑竟是我自己~~
   - 若类中无移动函数，`std::move` 不会失效，因为拷贝函数的形参是常量左值引用 `const T&`，而常量左值引用可以接收任何形式的实参（详见引用绑定规则），包括 `std::move` 返回的右值引用 `T&&`，所以此时会调用拷贝函数，而不会因为移动函数的缺失而报错，此现象遵从 `C++` 的**标准降级规则（Fallback）**。

# 三、编译器隐式自动合成规则表格

| 特殊成员函数 | 自动合成条件 | 不再自动合成的情形 |
| --- | --- | --- |
| 默认构造函数 | 未声明任何构造函数 | 声明任意构造函数（有参/拷贝/移动） |
| 析构函数 | 未声明析构函数 | 手动声明析构函数 |

| 特殊成员函数 | 自动合成条件 | 不再自动合成的情形 |
| --- | --- | --- |
| 拷贝构造函数 | 未声明拷贝构造、任何移动操作 | 声明拷贝构造/任意移动操作 |
| 拷贝赋值运算符 | 未声明拷贝赋值、任何移动操作 | 声明拷贝赋值/任意移动操作 |
| 移动构造函数 | 未声明拷贝操作、移动操作、析构 | 声明拷贝操作/移动操作/析构 |
| 移动赋值运算符 | 未声明拷贝操作、移动操作、析构 | 声明拷贝操作/移动操作/析构 |

> 注：表格中
> 、代表且
> / 代表或

# 四、Rule of Five（五法则）

如果一个类需要**手动显式实现**以下任意一个函数，为保证资源安全与行为一致，应当手动实现全部五个：

- 析构函数
- 拷贝构造函数
- 拷贝赋值运算符
- 移动构造函数
- 移动赋值运算符

## Rule of Five 的来源与提出者

Rule of Five 由 **Howard Hinnant**（C++ 标准委员会成员，移动语义核心设计者）在 C++11 标准化过程中正式提出，用于完善 C++98 时代的 Rule of Three（三法则），适配新增的移动语义。同时，这一原则为所有C++开发者规避内存问题提供了意义深远的指导。

# 五、极简记忆

- 默认构造：写任意构造即消失
- 析构：写了就不合成
- 拷贝：写自身或移动即失效
- 移动：碰拷贝/移动/析构任意一个就不生成

- 资源类：遵循 Rule of Five